



Project Proposal – Week 08

**Section:
BCS-3D**

**NU ID's:
24K-0527 24P-0582 24K-0826**

October 17th, 2025

—

**Syllabus:
Data Structures Theory**

Syllabus Code: CS-2001

—

**Instructor:
Sir Dr. Farrukh Hassan Syed**

Mini Instagram/Social Media Application

Brief Description:

The project idea revolves around emulation of the backend of how many of the social media platforms that we see as of today's date operate via the fundamental data structures -that we have covered in the course- and how the usage of these data structures provides us with the crucial features and makes the tasks such as (direct messaging, posting, mutual follower suggestions etc) that attract the majority of the user base and makes these apps more appealing to the users. This will also allow us to observe how all of these data structures differentiate in posing some restrictions and a certain set of rules on how their data can be accessed and manipulated and why these rules make the usage of these data structures all more appealing in such real-life applications and scenarios. As using them allows the developer to incorporate certain algorithms on certain data structures that not only optimize a particular task, but in some cases even make some operations possible on data that would not have been possible given the developer had opted for using a primitive (pre-defined) data structure instead. The project is centered around and uses the social media application Instagram as our reference. In a nutshell the main objective is to demonstrate how a social media platform utilizes data structures whilst handling various functionalities including user management, messaging, friend requests, posts and notifications.

Tentative list of data structures and algorithms that will be used:

1. User Management

- Description: Handles user registration, login, logout, and profile storage.
- Data Structures: Linked List, Hash Map
- Algorithms: Hashing (for username lookup), Linear Search (for Linked List traversal)
- Details:
 - o Each user node contains username, password, city, posts, and status.
 - o Uses hash lookup for fast authentication.
 - o Supports password reset and logout.

2. Relationship Graph (Followers/Friends, Likes, and Location of users)

- Description: Represents all users and their connections.
- Data Structures: Graph (Adjacency List)
- Algorithms: BFS (Breadth-First Search), DFS (Depth-First Search), Mutual friend detection (common neighbor search)
- Details:
 - o Each user node maintains a list of connected users.
 - o Edges represent relationships (followed/follower).
 - o Used for suggesting mutual friends or displaying connections.

3. Friend Requests

- Description: Manages sending and accepting friend/follow requests.
- Data Structures: Queue
- Algorithms: Enqueue /Dequeue Operations (FIFO), Traversal for pending request checks
- Details:
 - o Friend requests are stored in FIFO order.
 - o Supports viewing pending requests, accepting, and rejecting them.

4. Posts System

- Description: Allows users to create and view posts.
- Data Structures: Stack
- Algorithms: Push/pop for post creation and deletion Reverse traversal for recent-first feed
- Details:
 - o Each user maintains a stack of their posts.
 - o Latest post appears at the top for quick access.
 - o Supports basic timeline view

5. Feed Display

- Description: Shows a combined feed from all followed users.
- Data Structures: Stack + Traversal Logic
- Algorithms: Stack merge algorithm (combining feeds) sorting by timestamp
- Details:
 - o Collects posts from followers and merges their stacks.
 - o Displays most recent posts first.

6. Messaging System

- Description: Enables one-to-one chatting between connected users.
- Data Structures: Stack
- Algorithms: Push/pop for message order, Reverse iteration for message history display
- Details:
 - o Each chat has its own message stack.
 - o New messages are pushed to the top.
 - o Latest conversation retrieval is $O(1)$.

7. Notifications System

- Description: Stores alerts for user actions like friend requests or messages.
- Data Structures: Queue
- Algorithms: Enqueue/Dequeue operations , chronological traversal
- Details:
 - o Notifications are added in order of arrival.
 - o Users can view notifications in the order they occurred.

8. Search Users

- Description: Allows finding users quickly by name.
- Data Structures: Binary Search Tree (BST)
- Algorithms: Binary search, In-Order traversal (for alphabetical listing)
- Details:
 - o Users are stored alphabetically in BST.
 - o Searching is $O(\log n)$.
 - o Supports in-order traversal for sorted display.

9. Follower and Following Lists

- Description: Displays who a user follows and who follows them.
- Data Structures: Linked List
- Details:
 - o Each user node maintains two linked lists (followers & following).
 - o Can show total connections and mutual friends.

10. Notifications for Requests and Messages

- Description: Notifies users when a request is received or accepted.
- Data Structures: Queue
- Algorithms: Traversal (for display), Counting to complete followers/following totals)
- Details:
 - o Integrates with Friend Request and Messaging modules.
 - o Displays unread notifications first.

11. Active Status Tracking

- Description: Displays last login and active/inactive state.
- Data Structures: Hash Map / Timestamp Variable
- Algorithms: Hash Lookup (for user activity), time comparison (for offline/online detection)
- Details:
 - o Updates user status upon login/logout.
 - o Used for showing “last seen” and online indicators.